

# **Predictive Maintenance for Industrial Equipment**

Failure-window prediction, risk scoring and alerting from sensor data

Project report

Repository: [github.com/Puneeth7104/Predictive-Maintenance-for-Industrial-Equipment](https://github.com/Puneeth7104/Predictive-Maintenance-for-Industrial-Equipment)

## 1. Summary

Unplanned machine failures stop production and are expensive. This project builds a system that looks at the sensor readings and maintenance history of each machine and estimates the probability that the machine will fail within the next 24 hours. High-risk machines are shown on a monitoring dashboard and trigger Slack or e-mail alerts, so that maintenance can be planned before the breakdown.

**Key results on the untouched test period:** the selected model (Gradient Boosting (balanced)) reaches a precision of 0.92, a recall of 0.65 (per hourly reading), a PR-AUC of 0.78 and a ROC-AUC of 0.95. At the level of real failure events, an alert was raised before 22 of 22 failures, on average 16 hours ahead, with only 2 false-alarm episodes in about six weeks across 20 machines.

**Important note:** no real dataset was available, so all data is synthetic. The numbers therefore show that the pipeline works correctly, not how it would perform on a specific real plant.

## 2. Problem definition

Target: for every hourly reading of a machine, predict whether that machine will fail within the next 24 hours (label 1) or not (label 0). Failures are rare (about 2.5% of readings are positive), so accuracy is meaningless: a model that always predicts "no failure" would be 97% accurate and completely useless. We therefore evaluate with precision, recall, PR-AUC and the number of real failures that were warned about.

## 3. Data

Because no dataset was supplied, `src/generate_data.py` simulates 20 machines (pumps, compressors and motors) with hourly readings for 180 days.

| Item                    | Description                                                                                                                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sensor columns          | temperature, vibration, pressure (hourly)                                                                                                                                                                   |
| Usage / history columns | operating_hours, hours_since_maintenance, maintenance_count                                                                                                                                                 |
| Maintenance log         | timestamp, machine, event type (failure or preventive)                                                                                                                                                      |
| Size                    | 85,131 sensor rows, 89 failures, 20 machines                                                                                                                                                                |
| Realism                 | gradual degradation before failures (48-96 h), 20% sudden failures (about 10 h warning), 25% of cycles end with planned maintenance, random vibration spikes, 0.5% missing readings, 8-16 h repair downtime |

Before a failure, temperature and vibration rise and pressure falls. Figure 1 shows the average drift of each sensor in the 120 hours before a failure; the dashed line marks the 24-hour prediction window.

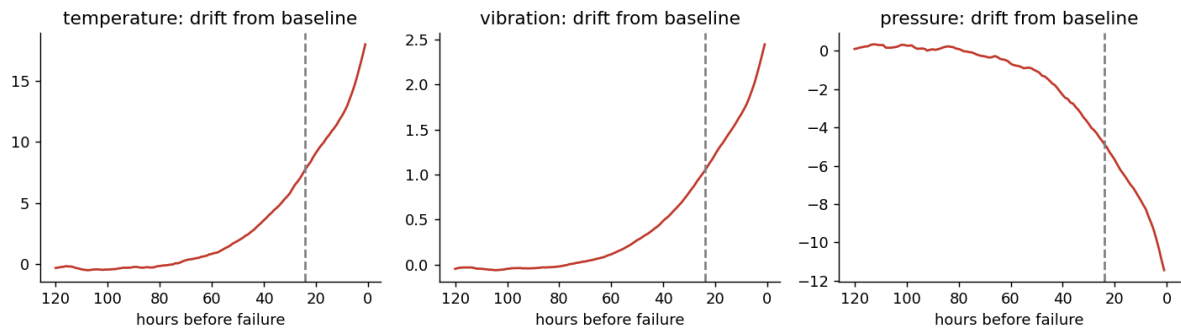


Figure 1 - Average sensor drift from the machine baseline before failures

## 4. Method

### 4.1 Feature engineering without leakage

All features at time  $t$  use only readings from time  $t$  and earlier. Missing readings are forward-filled (last known value). For each of the three sensors we computed:

- rolling mean and standard deviation over 6 h and 24 h, plus rolling maximum and minimum over 24 h;
- lag features: change versus 6 h and 24 h earlier;
- drift from the machine's own normal level: the 6 h mean minus the expanding (past-only) average of that machine.

Together with the raw sensor values, hours since last maintenance and the maintenance count, this gives 32 features. A unit test (tests/test\_pipeline.py) proves that removing future data does not change any feature value, which is the practical check for leakage.

### 4.2 Failure window and label

A reading is labelled 1 if the machine's next failure occurs within 24 hours after it. Future failure times are used only to create this label, never as features.

### 4.3 Chronological split

Random splitting would let the model see the future, so the data is split by time: the first 60% for training, the next 15% for validation (model choice and alert threshold) and the last 25% for the final test. A 24-hour gap between the parts prevents labels from overlapping. Rows: train 50,800, validation 12,209, test 20,694 (test period 2026-05-17 to 2026-06-29).

### 4.4 Handling rare failures

Two approaches were compared: class weights (the loss function penalises a missed failure more) and random undersampling of normal readings to a 5:1 ratio. An unweighted logistic regression serves as the baseline.

### 4.5 Models and threshold

Five models were trained: logistic regression (with and without weights), random forest (class-weighted and undersampled) and gradient boosting (class-weighted). The model with the best validation PR-AUC

was selected. The alert threshold is the highest-precision threshold on the validation set that still reaches 80.0% recall, so the alert level reflects the cost of missing a failure.

## 5. Results

### 5.1 Model comparison

| Model                            | Val PR-AUC | Test PR-AUC | Precision | Recall | F1   | ROC-AUC |
|----------------------------------|------------|-------------|-----------|--------|------|---------|
| Logistic (no weights)            | 0.937      | 0.794       | 0.91      | 0.67   | 0.77 | 0.956   |
| Logistic (balanced)              | 0.903      | 0.767       | 0.80      | 0.65   | 0.72 | 0.965   |
| Random Forest (balanced)         | 0.932      | 0.790       | 0.93      | 0.66   | 0.77 | 0.951   |
| Random Forest (undersampled 5:1) | 0.927      | 0.810       | 0.92      | 0.67   | 0.78 | 0.965   |
| Gradient Boosting (balanced)     | 0.939      | 0.780       | 0.92      | 0.65   | 0.76 | 0.951   |

The validation PR-AUC values of all models are close (0.90-0.94). The differences on the test set are small as well, so the choice between these models is not decisive; with only 22 test failures such small gaps could be noise. Gradient boosting was selected strictly by the validation rule defined beforehand. The unweighted logistic regression is a strong baseline because the degradation signal in the synthetic data is clear; on real data a bigger gap between simple and complex models is more likely.

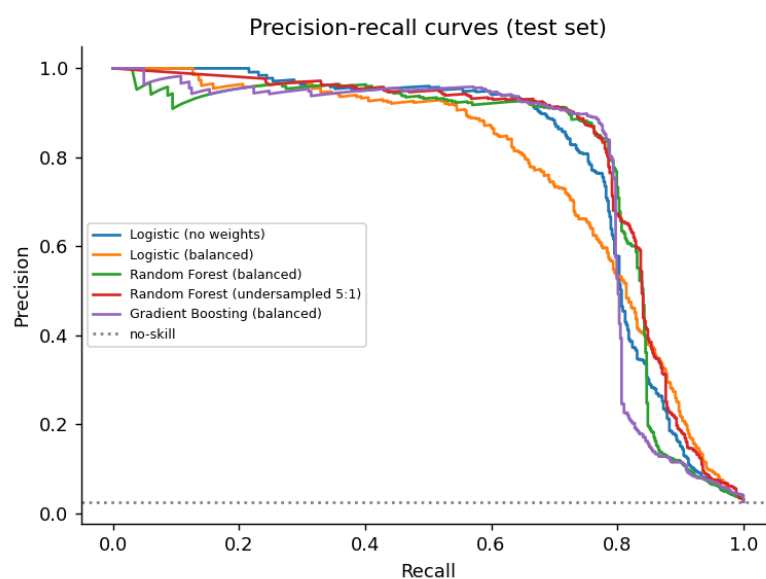


Figure 2 - Precision-recall curves of all models on the test set

### 5.2 Selected model on the test set

| Metric                   | Value |
|--------------------------|-------|
| Precision                | 0.925 |
| Recall (hourly readings) | 0.650 |
| F1                       | 0.763 |
| PR-AUC                   | 0.780 |

| Metric                                | Value                  |
|---------------------------------------|------------------------|
| ROC-AUC                               | 0.951                  |
| Alert threshold                       | 0.998                  |
| Failures with an alert before failure | 22 of 22               |
| Mean warning time                     | 16.4 hours             |
| False-alarm episodes (test period)    | 2 of 24 alert episodes |
| False alarms per machine per week     | 0.016                  |

Hour by hour, the confusion matrix shows 343 correctly flagged at-risk readings, 28 false alarms, 185 missed at-risk readings and 20,138 correctly ignored normal readings. A mean warning time of about 16 hours out of a possible 24 suggests that many misses are early hours of a warning period, where the alert fires later but still before the failure (not analysed separately). This is why the event-level view (every failure was warned about, on average about 16 hours ahead) is more useful for maintenance planning than the hourly recall.

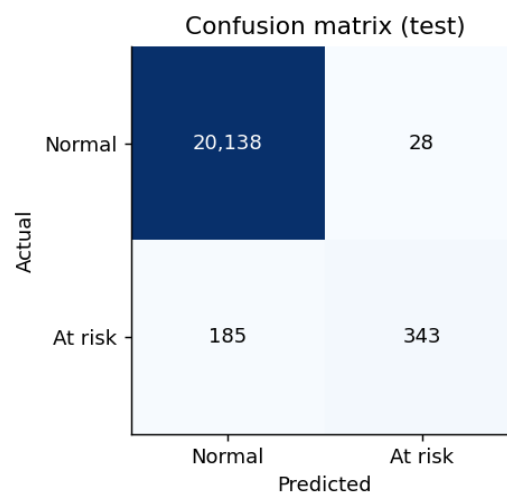


Figure 3 - Confusion matrix on the test set (hourly readings)

**Honest caveat:** the hourly recall of 0.65 is below the 0.80 reached on the validation data. The threshold was tuned on older data, and sudden failures (about 10 hours of warning) are a likely contributor, because they leave little time to raise the risk score (this was not analysed separately). A lower threshold would raise recall at the cost of more false alarms; the threshold can be changed in `src/train.py` (RECALL\_TARGET in `src/config.py`).

### 5.3 What the model relies on

| Rank | Feature                 | Importance (PR-AUC drop) |
|------|-------------------------|--------------------------|
| 1    | vibration_std_24        | 0.0265                   |
| 2    | vibration_diff_24       | 0.0122                   |
| 3    | vibration               | 0.0117                   |
| 4    | vibration_dev_baseline  | 0.0093                   |
| 5    | hours_since_maintenance | 0.0089                   |

| Rank | Feature             | Importance (PR-AUC drop) |
|------|---------------------|--------------------------|
| 6    | temperature_diff_24 | 0.0046                   |
| 7    | vibration_diff_6    | 0.0027                   |
| 8    | vibration_min_24    | 0.0026                   |

Vibration-based features dominate, matching how the data was generated and matching common practice in condition monitoring, where vibration is an early indicator of bearing and shaft problems. Hours since maintenance also contributes, which shows that the maintenance history adds information.

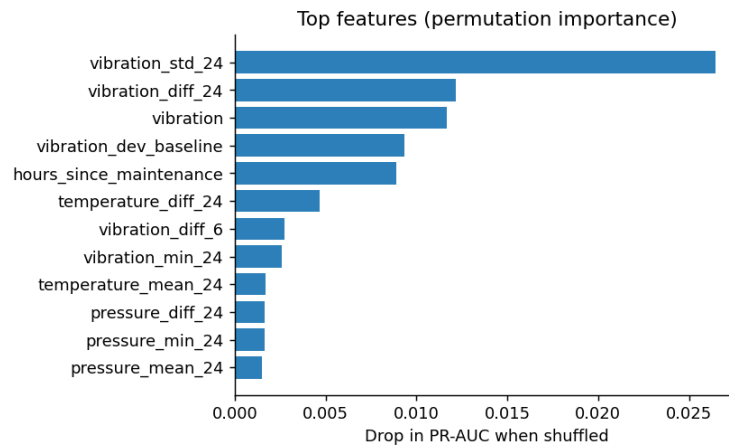


Figure 4 - Permutation importance on the validation set

## 5.4 Example: a risk score before a failure

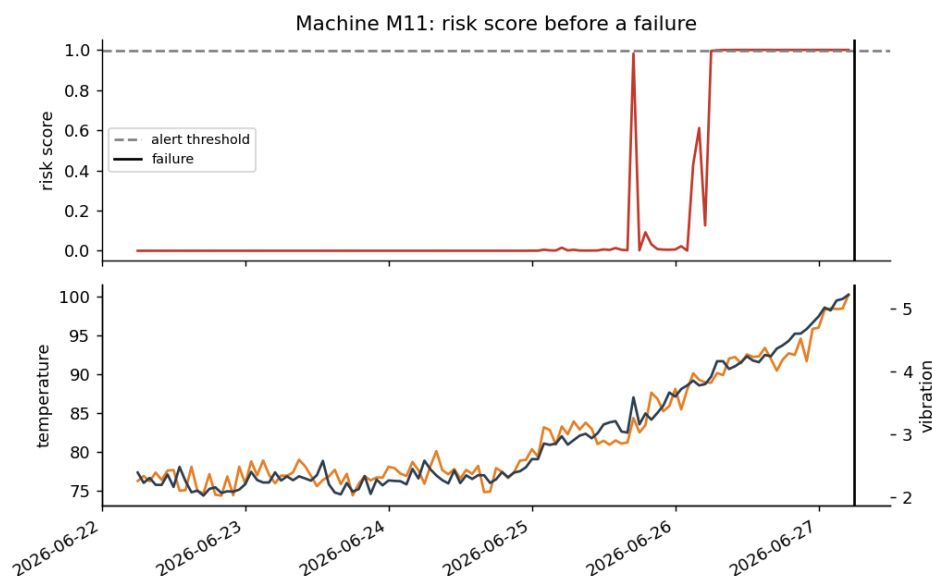


Figure 5 - Risk score and sensor values for one machine before a failure

## 6. Monitoring and alerting

src/risk\_scoring.py scores every machine with the saved model and writes the latest risk score per machine, a 14-day risk history and the main driver of the risk (the sensor that deviates most from its baseline, in standard deviations). Machines are grouped as High (score above the alert threshold),

Medium (above half of the threshold) and Low. Machines without a reading for 3 hours are shown as stopped because they are in repair or maintenance.

The same results are presented in two ways:

- **Streamlit app (app.py):** fleet overview with risk bars, a machine detail page with risk and sensor charts, model performance with figures, and an alerts page with a "Send alerts now" button.
- **Static dashboard (docs/index.html):** a single self-contained page for GitHub Pages with a sortable risk table and a clickable risk-history chart.

Alerting (src/alerts.py) sends a message listing every high-risk machine, its risk score, the main driver and the hours since the last maintenance. It supports a Slack incoming webhook and e-mail through SMTP. When no credentials are configured it runs in dry-run mode and only logs the message to data/alerts\_log.csv, so it can be tested safely. A GitHub Actions workflow (alerts.yml) can run it on a schedule using repository secrets.

## 7. Deployment

- Code, data, trained model, figures and dashboards are stored in the GitHub repository.
- GitHub Actions (ci.yml) runs the automated tests on every push.
- Streamlit Community Cloud deploys app.py directly from the repository; if the saved model cannot be loaded because of a different scikit-learn version, it is retrained automatically on first start.
- GitHub Pages publishes the /docs folder of the main branch as a public dashboard.
- Secrets (Slack webhook, SMTP password) are never committed; they are provided through Streamlit secrets or GitHub secrets.

## 8. Limitations and future work

- The data is synthetic. The pipeline must be re-run on real sensor history, and the thresholds re-tuned, before real decisions depend on it.
- The test period contains only 22 failures, so the event-level results have a wide uncertainty; a longer history or cross-validation over time (rolling-origin) would give firmer numbers.
- The dashboard replays the last 14 days of the dataset instead of reading a live sensor stream. A production version would add a data ingestion service (for example MQTT or a database) and a scheduled scoring job.
- Possible improvements: remaining-useful-life regression, probability calibration, per-machine thresholds, cost-based threshold selection (cost of a missed failure versus an unnecessary inspection), and drift monitoring.

## 9. How to reproduce

Clone the repository, create a virtual environment, run `pip install -r requirements.txt`, then `python run_pipeline.py` to regenerate data, model, risk scores and the static dashboard, and `streamlit run`

app.py to start the app. Random seeds are fixed (seed 42), so the results in this report can be reproduced exactly with the same package versions.